1. Introduction

RAG RetrievalAugmented Generation is a technique that mixes two ideas:

1. **Retrieval** pulling out relevant pieces of information (usually from a large text database or knowledge base).
2. **Generation** letting a language model (e.g., GPT, LLaMA) write a response in its own words.

Instead of asking the model to guess everything from its internal knowledge, RAG first fetches the most useful facts and then asks the model to incorporate them. The result is usually more accurate, uptodate, and grounded in real data.

---

2. Explanation

1. **User query** What is the timecomplexity of quicksort in the worst case?
2. **Retriever** searches a collection of documents (Wikipedia, PDFs, code repos, etc.) and returns the topk passages that look relevant.
3. **Generator** receives the original query **plus** those passages as context and produces a final answer.

The generator can be a standard LLM, but because it now sees concrete evidence, it can:

* Cite sources.
* Avoid hallucinations (making up facts).

* Answer questions about very recent events that the model itself was never trained on.

---

3. Subtopics

Subtopic	What it means	Why it matters
Retriever models	BM25, dense vector search (e.g., FAISS, Elastic, DPR), hybrid methods.	Determines how well the right documents are found.
Indexer / Knowledge base	The data store that holds the raw texts (Wikipedia dump, PDFs, code, logs).	Quality & freshness of the index directly affect answer quality.
Chunking & Passage selection	Splitting long documents into manageable pieces (e.g., 200token chunks) and picking the topk.	Keeps the context size within the LLMs token limit.
Fusion strategies	*Late fusion*: retrieve generate. *Early fusion*: embed retrieved text directly into the prompt. *Hybrid*: add a knowledgeselector layer.	Different ways to combine retrieval and generation tradeoff speed vs. accuracy.
Prompt engineering for RAG	Designing the prompt that tells the model how to use the retrieved passages (e.g., Use the following sources to answer).	Guides the model to cite and not ignore the evidence.
Evaluation metrics	ExactMatch, F1, ROUGE, BLEU, and **groundedness** (does the answer follow the retrieved text?).	Measures whether RAG actually improves over plain generation.
Finetuning vs. Promptonly RAG	Finetune the LLM on retrievalaugmented data vs. only change the prompt.	Finetuning can give higher performance but costs more compute.
Safety & Bias	Retrieval can bring in toxic or biased passages; the generator must filter them.	Prevents harmful outputs.

---

4. Example

Scenario: A student asks a chatbot, Explain the difference between TCP and UDP.

Stepbystep RAG flow

Step	Action	Result
1	**Query**	Explain the difference between TCP and UDP.
2	**Retriever** searches a prebuilt index of networking textbooks & RFCs.	Returns two passages: 1. RFC793 excerpt on TCP reliability. 2. RFC768 excerpt on UDP simplicity.
3	**Prompt construction**	The system builds a prompt: ```\nYou are a helpful tutor. Use the following sources to answer the question.\nSources:\n[1] ... (TCP passage) ...\n[2] ... (UDP passage) ...\nQuestion: Explain the difference between TCP and UDP.\n```
4	**Generator (LLM)** creates a response, citing the sources.	TCP is connectionoriented and guarantees delivery (source[1]). UDP is connectionless and does not guarantee delivery (source[2]).
5	**Output**	The student receives a concise, sourced answer.

What changes compared to a plain LLM?

- The answer is grounded in the actual RFC text, not just the models memory.
- The model can point to the exact source IDs, making verification easy.

---

5. Key Points

	Point
Grounding	RAG ties the models output to real documents, reducing hallucinations.
Freshness	By updating the knowledge base, you can answer about events after the LLMs training cutoff.
Scalability	Retrieval can handle billions of tokens with vector indexes, far more than any LLMs context window.
Prompt design matters	Clear instructions (cite the sources) help the generator use the retrieved info correctly.
Hybrid systems	The best solutions often blend RAG with a small amount of finetuning or tooluse (e.g., calculators).
Performance tradeoff	Retrieval adds latency (search time) but usually improves accuracy.
Safety	Bad or biased documents can be retrieved; you need filters or postprocessing.
Evaluation	Look at both answer quality *and* how well the answer follows the retrieved evidence.

---

6. Sources

1. **Lewis, Patrick et al.** RetrievalAugmented Generation for KnowledgeIntensive NLP Tasks. *Proceedings of NeurIPS 2020*.

2. **Karpukhin, Vladimir et al.** Dense Passage Retrieval for OpenDomain Question Answering. *EMNLP 2020*.

3. **Gao, Tianyu et al.** RetrievalAugmented Language Model Finetuning. *ACL 2022*.

4. **OpenAI** RetrievalAugmented Generation (RAG) in ChatGPT plugins. Blog post, 2023.

5. **Microsoft Research** The RAG Paradigm: Combining Search and Generation. Technical Report, 2022.

(All links are publicly accessible; you can find PDFs on arXiv or the conference websites.)

---

7. Simple Comparison Table

Concept	How it works	Typical use case	Strength	Weakness
Plain Generation (e.g., GPT4 alone)	Model generates from its internal weights only.	Creative writing, brainstorming.	Fast, no external data needed.	Can hallucinate; outdated knowledge.
Finetuned Model	Model is further trained on a specific domain (e.g., medical QA).	Domainspecific assistants.	Better on that domain; no extra retrieval step.	Expensive to train; still limited to training cutoff.
RetrievalOnly (search engine)	Returns documents, no generation.	Bibliographic search, factlookup.	Precise citations, no hallucination.	User must read and synthesize; no natural language answer.
RAG (RetrievalAugmented Generation)	Retrieve docs feed them to LLM generate answer.	Customer support, uptodate QA, tutoring.	Grounded, uptodate, concise answers.	Extra latency; needs good indexing & filtering.

| **ToolUsing LLM** (e.g., code execution, calculator) | LLM decides to call external tools and incorporate results. | Math solving, datavisualisation. | Exact computation, no approximation. | More complex orchestration; tool security concerns. |

The table is intentionally kept short and uses plain language so a 3rdyear computerscience student can quickly see the tradeoffs.